Code Inspection Report

For: Thread Zeppelin — Digital Thread Summarization Tool

By: PenUltimate

Brett Palmer, Monica Agneta, George Pitt, Rebecca Sonnemann, Ben Caras

11 March 2026

Table of Contents

1. Introduction 2
    1.1 Purpose of This Document 2
    1.2 References 2
    1.3 Defect Checklist 3
2. Summary of the Incident 4
3. Detection, Reporting, and Initial Response 6
    3.1 Detection 6
    3.2 Timeline of Events 7
    3.3 Impact Assessment 8
4. Modules Inspected 9
    4.1 Defects Identified 10
    4.2 Contributing and Mitigating Factors 13
5. Unified Reporting Assessment and Analysis 14
    5.1 Root Cause 14
    5.2 Risks Identified 15
    5.3 Follow-Up Actions 16
6. Outstanding Tasks 16
Appendix A: Team Review Sign-Off 18
Appendix B: Contributions 19

1. Introduction

This document presents an incident report of the Thread Zeppelin computer science capstone project developed for the University of Maine's Advanced Structures and Composites Center (ASCC). The original project request was to develop a digital thread summarization tool that would analyze manufacturing data and generate summaries using a large language model (LLM). Throughout the project's execution, the team and client faced increasing conflict regarding the suitability of LLMs for the type of structured and unstructured data being analyzed. This disagreement ultimately resulted in the project reaching a halt in development. This report was prepared by Thread Zeppelin's sibling team that was responsible for reviewing the incident, assessing project artifacts, and analyzing the technical and collaborative factors that contributed to the situation.

1.1 Purpose of This Document

The purpose of this document is to document and analyze the circumstances that led to the disruption of the Thread Zeppelin capstone project. This report combines a traditional post-mortem analysis with a partial inspection of project artifacts related to the incident. It provides a structured examination of the incident by summarizing the project timeline, identifying defects in both the development process and team collaboration, and evaluating the technical and organizational factors that contributed to the outcome. In addition, the document suggests alternative solutions that could have been attempted, assesses the incident's impact on the client, and provides recommendations for future work given the current situation. The overall goal is to provide a clear record of what occurred, identify lessons that the team can take from this experience, and offer insights that could prevent future capstone students from encountering a similar issue when working with their client.

1.2 References

The following documents and materials were reviewed during the investigation process. These sources provide context on the team's project timeline, technical decisions, and communications with the ASCC.

Table 1 — References

Source Material Name	**Description**
Code Inspection Discussion	Provided background context on the incident that occurred within Thread Zepellin's capstone project
Project Timeline Document	A timeline created by Thread Zepellin summarizing key

	milestones, meetings, and notable events throughout the project lifecycle
Client Meeting Notes and Transcripts	Records of meetings between the team and client, including discussion summaries, diagrams, and architectural explanations provided by the client
LLM Training Source Files	Ten source files and results provided by Ben Miller, who attempted to train and configure an LLM for the project
Project Proposal	The original project proposal containing project goals, assumptions, and recommended technical approach
Software Requirements Specification (SRS)	Document of the intended system requirements and functional expectations as agreed upon by the team and client
System Design Document (SDD)	Outline of the planned system architecture and technical approach as agreed upon by the team and client
User Interface Design Document (UIDD)	Document that presents the intended design of the UI components as agreed upon by the team and client
Critical Design Review Document (CDRD)	Material that summarizes the project's progression over its first three months

1.3 Defect Checklist

For this incident report, we have categorized defects by describing mistakes in the Thread Zepellin's team structure and behavior rather than by code-specific flaws. Table 2 names these defect types and provides a description of each.

Table 2 — Defect Types

Defect Category	**Defect Type Description**	**Defect Type Severity**
Acceptance of Flawed Proposal	Agreeing to take on a project whose proposed technical approach contained questionable assumptions	High
Incomplete Requirement Verification	Failing to clarify ambiguous requirements with the client in the design stage of the project	High
Delayed	Waiting too long to consult course staff when	High

Escalation of Concerns	encountering major project risks or uncertainty	
Reactive Risk Management	Recognizing major project risks only after significant time has already been spent on an ineffective approach	High
Overreliance on External Dependencies	Relying heavily on external tools or datasets without contingency plans in case those resources are unavailable	High
Limited Exploration of Alternative Solutions	Telling the client that alternative approaches are more promising without attempting them as proof	Medium
Insufficient Evidence for Concerns	Raising concerns about the feasibility of the project without formal research or analysis to support the claims	Medium
Information Siloing	Allocating crucial technical work to a single team member instead of being collaboratively building the knowledge	Medium
Limited Stakeholder Communication	Struggling to maintain consistent, structured meetings with the client to regularly address questions and concerns	Medium
Disconnect with Experts	Meeting only with the client, instead of discussing the tool with specialists who better understood the technology	Medium
Poor Internal Communication	Failing to reach consensus for team decisions	Low

2. Summary of the Incident

This section provides a high-level overview of the incident, including the timeline, impact, and the resolution that followed.

Incident Title: Requirement Misalignment with LLM Capability for Data Summarization Tool

Date and Time of Impact

On November 14, 2025 the Capstone team concluded that an LLM was not fit for the proposed project.

Duration: 4 months

Severity

High severity as the capstone project prototype was delayed and the original proposed requirements were not achievable with the given constraints.

Affected Users/Systems

The affected users include Thread Zeppelin's development team working on the project, the client expecting a data summarization tool to be made with LLM ready for use by May 2026, and the end users of the Advanced Structure and Composites Center.

Resolution Summary

After careful consideration the suggested resolution is for Thread Zeppelin's capstone project to transition into a research focused project that involves evaluating multiple LLM models and training methods for summarizing large amounts of manufacturing data. The original proposed project presented several technical constraints and unclear requirements which created challenges and misalignment between the client and the development team. The Thread Zeppelin team spent several months attempting to locally train the recommended LLM but feasibility issues prevented the model from being successfully implemented as the intended tool. After consultation with LLM experts and further testing it was concluded the original approach would need to be significantly reworked.

As a project alternative, the team will produce a comprehensive research paper evaluating the three recommended LLM models: GPToSS, Quen 3, and Nemotron 3 along with their training strategies. Since the client is part of the Advanced Structures and Composites Center, this research could provide valuable insights for future development of a data summarization tool within the center. The research findings may help to guide a more feasible approach for the originally proposed tool at a later time. This alternative project also benefits the Thread Zeppelin development team by allowing them to gain further experience with LLM models, which was the main motivation for selecting the project. Additionally, the team will be able to focus on structured technical evaluation rather than continuing efforts that were constrained by resource limitations.

The research will aim to identify a viable alternative method for addressing the ASCC's manufacturing data summarization problem. Based on the findings, the development team may then revise their original project proposal to incorporate the identified best reworked approach alongside the research paper.

3. Detection, Reporting, and Initial Response

The issues in the project were developed gradually over the course of the semester due to ongoing communication issues between the team and the client. Early in the project, the client expressed a strong preference for using the large language model o-Llama 3 to analyze manufacturing data and generate summaries.

As development progressed, the team began to recognize inconsistencies between the project requirements and the technical approach being requested. While the team raised these concerns internally and occasionally with the client, the issue was not escalated until later in the first semester and in the beginning of the second semester.

Because of this, the team continued attempting to develop an LLM-based solution even though there was uncertainty about whether the approach was appropriate. Additional challenges came up due to limited access to realistic datasets, incomplete documentation of the data fields, and an inability to communicate directly with the system's users.

3.1 Timeline of Events

Detection Method

The issue was primarily detected through internal discussions among team members as development progressed. Team members noticed inconsistencies between the technical approach requested by the client and the actual analysis being performed on the data. These concerns were reinforced during later meetings when the AI expert reviewed the project and suggested that the specified approach might not be appropriate.

Detection Timestamp

Early signs of the issue appeared during their team discussions in mid-November 2025, when the team began questioning whether an LLM was necessary for the tasks being requested.

First Response Timestamp

The first response occurred shortly after these discussions when the team considered discussing the issues with the course staff. However, the issue was not fully addressed at that time, and development continued using the same general approach.

3.2 Timeline of Events

The timeline in Table 3 outlines the major events that occurred throughout Thread Zeppelin's project. Reviewing the steps in chronological order could point to where issues started.

Table 3 — Timeline of Events

Date	Description
10/1/25	- Initial introduction email sent to client.
10/10/25	- Initial meeting with their client. - Initial request of datasets, data formats, and summarization level (operational, executive, etc), and expected output.
10/16/25	- Requested extension on their SRS due to difficulty with soliciting requirements from their client.
10/19/25	- Extension on their SRS got denied.
10/24/25	- Client and Team Meeting. - Told to use Llama by their client.
11/11/25	- Received sample data
11/12/25	- Team Meeting - The team discussed being confused about why they needed to use an LLM. - Asked at what point they see Professor Yoo about potentially changing projects since they were having trouble.
11/14/25	- Meeting with the team and client. - Brought up the fact they are unsure of the point of having an LLM do basic math. - Client drew out how she would like everything to connect and asked if they could have an MCP server handling the threading of potentially using multiple LLMs with 1 doing the math and 1 writing the summary.
12/18/25	- Initial GitHub is up and running on two machines.
12/19/25	- UI draft is completed.
1/26/26	- Llama 3 was installed and is running on Ben Miller's computer.
2/2/26	- Found additional test data in their Gitlab however they are unsure if it is up to date.
2/4/26	- Told them that they are only analyzing CSV files in contrast to different types of text based files in fall semester.
2/5/26	- Emailed Oz (AI specialist that built the initial website they are building off of) to try to obtain more mock data. Oz responded he did not have any data he was able to share but wanted to discuss with the client to see if he could get them more than

	they already had. Gave them a few recurring fields that occur between prints that they may want to include. - Asked Oz to attend the next meeting.
2/18/26	- Had a meeting with their client, the team (minus 1), and Oz. - Oz brought up the fact Llama is not good at doing this type of work and recommended they look into GPToSS, Quen 3, and Nemotron 3 - Scheduled a meeting with Johnny. - Within 5 days of meeting with Oz, Ben began attempting to train GPToSS.
2/23/26	- Client email to move our meeting due to a snowstorm.
2/24/26	- Had a meeting with 3 team members, their client, and ASCC AI specialist. The specialist attempted to walk them through how to train the AI but also was unsure how they could proceed without having additional training data and was unsure how they should do a summary if they do not have interviews or information from end users. - They attempted to merge UI changes from December. Found there was an issue with containerization and the move from their Gitlab to our Github. They didn't have access to their Gitlab so they couldn't push any changes to any submodule.
2/25/26	- Team meeting with Johnny. Johnny initially discussed not using the LLM and just doing it via functions. Discussed some potential sections for poster including: - How WOULD you implement an LLM-based summarizer? - LLM's in data analysis. What are its limitations? - Different LLM's and their different applications. - How do you train an AI well? - Met with Johnny after class where Johnny said to stop developing and transition to doing a research based project.
3/2/26	- Met as a team to discuss potential research areas and the potential of doing an experiment based research project by testing 2 types of training on 3 models types to compare hallucination rate, precision, and amount of data covered.
3/4/26	- Met with PenUltimate for CIR.

3.3 Impact Assessment

This section evaluates how the issues described earlier affected the project and the team's progress. It examines the impact of communication with the client, internal team coordination, and the overall development process.

Frequency of Meetings

The team met with the client about every other week throughout the semester. However, the meetings did not always result in clear direction or requirements. Meetings sometimes introduced changes to the project rather than clarifying existing expectations.

Meeting Quality

Most meetings included the client and several team members. Occasionally, they had AI experts join meetings to provide additional technical insight. The quality of meetings varied. While some meetings helped the team better understand the client's goals, others resulted in more confusion because of changing requirements or unclear expectations. The client's insistence on using an LLM despite the team's concerns contributed to uncertainty about the direction of development. The team also mentioned the client being 20 minutes late to one of their meetings.

Operational Impact

The team invested a significant amount of time attempting to satisfy requirements that were not clearly defined. This reduced the amount of time available for alternative solutions or revising the project earlier in the semester. The Advanced Structures and Composite Center was also affected throughout this project. They had a loss of time, a loss of money spent on tokens, and ended up not receiving the tool they wanted.

Customer Communications

Communication with the client was frequent but sometimes ineffective for them. Requirements changed multiple times, specifically about data formats and system capabilities. Additionally, the team was unable to obtain sufficient datasets or consult with AI experts early in the project. They also requested to meet with the employees who would be using the tool, but were unable to.

Internal Communication

Despite the external communication challenges with the client, internal communication within the team was generally effective. However, only one or two people seemed to be working on the development throughout the project, while the others worked on more of the documentation.

4. Modules Inspected

This section discusses the major modules relevant to the suspected areas of failure for Thread Zeppelins project. A module can be a system component or capability or it could be one of Thread Zeppelins documents from their project such as the Software Requirements

Specification or the System Design Document. Any modules from section 1.2 not listed are thought to not have been linked to the areas of failure or incident.

Client Meeting Notes and Transcripts

The client meeting notes and transcripts were included in the partial inspection due to the fact that the client disregarded team concerns regarding the use of an LLM. In multiple meetings Thread Zeppelin expressed concern to their client that an LLM is not capable of performing the calculations or functionality outlined by the client. This allowed for the issue to continue to remain unresolved until there were severe repercussions.

LLM Training Source Files

The LLM training source files is the training data used to train the LLM. The LLM training source files were included in the partial inspection due to the fact that the LLM was the point of failure for Thread Zeppelin. The use of an LLM made it impossible to achieve the desired functionality; only one team member was working with the LLM.

Software Requirements Specification (SRS)

The SRS serves as a blueprint for the systems performance, functionality, constraints, and requirements. The SRS was included in the partial inspection due to the mixed and unfulfilled requirements. The requirements outlined in this document made it impossible for Thread Zeppelin to complete their project given these constraints.

System Design Document (SDD)

The SDD defines how the system will be built in order to satisfy the requirements. The SDD was included in the partial inspection due to Thread Zeppelins' choice of an LLM. The use of the LLM made it impossible to achieve the desired functionality.

Critical Design Review Document (CDRD)

The CDRD validates the stability of the design and justifies the continuation of the project into code development, testing, etc. The CDRD was included in the partial inspection due to the fact that the team wrote the CDRD and moved into code development while they believed they were not able to complete the project given their clients constraints and requirements.

4.1 Defects Identified

This section lists all of the defects pertaining to the project of Thread Zeppelin. These defects could include components, files, and code from Thread Zeppelin’s application; defects could also be times when the team mismanaged their project or made a mistake in how they handled conflict resolution. Defects appear in the order from most critical to least critical.

Table 3 – Defect List

Defect Number	**Defect Type**	**Defect Description**	**Mistake**	**Recommendation**
1	Acceptance of Flawed Proposal	The project proposal was not reviewed by an LLM expert before being accepted.	The requirements for developing an LLM tool were agreed upon before proper consideration of constraints.	Ensure the client proposing the project is knowledgeable in what is being requested before beginning development**.**
2	Incomplete Requirement Verification	The CDRD described LLM selection as an "uncertainty" and also "one of the most important early steps."	Not using the SRS as a chance to find out what specifications the LLM would need to meet, and the SDD as a chance to research which tools meet these standards.	If making a revised proposal, elicit the core requirements for the LLM before even getting into the details of the technology. Once you have the standards, then research available tools.
3	Delayed Escalation of Concerns	The team didn't notify Professor Yoo about how the client was ignoring their concerns regarding the use of the LLM.	The team wasn't proactive and sought help before major problems started to arise.	Speak with either Professor Yoo or Johnny Sylvain and explain your concerns before major problems occur.
4	Reactive Risk Management	The team used an ineffective approach for an extended period of time before recognizing major flaws in their approach.	The team did not properly reflect on their approach early on, leading them to miss several major risks, such as the lack of access to the database, and minimal mock data available for training the LLM.	Lay out an overarching plan for the project early on in the development life cycle. Regularly identify risks with their likelihood and impact, and address those with the largest risk exposure.
5	Overreliance on External Dependencies	The team was not given the necessary information to develop the tool.	The team did not inquire with their client about which resources would be provided for training the LLM.	Speak to the client to identify and require the needed resources to be shared early on in the development life cycle.

			Instead, the team tried to work around mock data limitations.	
6	Limited Exploration of Alternative Solutions	The team argued that the client's problem could be solved with statistics instead of an LLM, but they did not attempt this approach.	The team wanted the client to consider an alternative solution, but had not demonstrated why it was worth consideration.	The team had great ideas for alternate solutions, but they should have created a demo to support their ideas.
7	Insufficient Evidence for Concerns	The team did not have proof that the client's proposal was impossible to achieve.	The team claimed that the client's proposal wouldn't be possible, but they didn't have any evidence.	The team should have researched the topic and gone to the client with proof that their proposal wouldn't work, if found.
8	Information Siloing	Ben Miller was doing the LLM training locally on his machine, without source control	Even for early experimentation, assigning only one person to a significant task causes information to be isolated on their local machine.	Have code be shared and collaborative during early stages so that flaws or requirement issues are more likely to be discovered.
9	Limited Stakeholder Communication	The team met with their client every two to three weeks, even at the peak of conflict.	The team met with the client infrequently, which allowed unresolved issues to build up into the spring semester.	The team should have requested to meet more frequently with the client, or at least with relevant ASCC members.
10	Disconnect with Experts	The team didn't meet with one of their clients' LLM experts until it was very late into the project.	The team was told that the client would get them in touch with one of their LLM experts but they didn't.	Continue to reach out to your client and be persistent about meeting this LLM expert.
11	Poor Internal Communication	3 out of 5 team members didn't agree on choosing this project.	The team didn't communicate their opinions on the project in the beginning.	Pick a project that all or most team members agree on, and communicate with other team members.

4.2 Contributing and Mitigating Factors

Tables 4 and 5 respectively list contributing and mitigating factors pertaining to the areas of incident or failure in regards to Thread Zeppelin's project.

Table 4 – Contributing Factors

Summary	Details
Thread Zeppelin didn't reach out to Johnny or Professor Yoo when problems first started to occur.	Thread Zeppelin had multiple problems or issues with their project and they didn't reach out to Johnny or Professor Yoo until they were major problems due to the fact that they were afraid of how they would be perceived by Professor Yoo. This prevented a resolution from being reached sooner.
The team's concerns were not taken seriously by the client.	The team repeatedly told their client that they believed they wouldn't be able to use an LLM to achieve their clients desired outcome. Their client ignored these concerns and told them that they must use an LLM to reach the specified output.
Only one person working with LLM and training data.	Ben Miller was training the LLM using the training data on his personal computer at home; no other group members had such significant interactions with the LLM.
The team didn't completely agree on this as their project.	All team members except for one wanted to choose a different project. Due to the fact that one person vetoed that project they subsequently chose this one.
Acceptance of flawed proposal.	The team wasn't aware that the proposal wasn't possible given the clients specifications as to how the system must function and perform.
The client didn't bring in an LLM expert when requested by the team.	Thread Zeppelin requested that their client introduce them to one of their LLM experts. The team didn't meet with one of their clients' LLM experts until a significant amount of time had passed after their initial request.

Table 5 – Mitigating Factors

Summary	Details
Receiving advice or input from an LLM expert.	The team expressed their concerns to their client that they are not able to use an LLM to perform the tasks and operations specified by the client. Thread Zeppelin and their client met with an LLM expert where this expert confirmed Thread Zeppelin's suspicions regarding the use of an LLM.
Told Johnny and Professor Yoo about the problems Thread Zeppelin has been having with their project and their client.	Thread Zeppelin informed Johnny and Professor Yoo that they are not able to complete their project to their clients' specifications. This helped set the project in a more beneficial direction.

5. Unified Reporting Assessment and Analysis

This section discusses what worked well, and what didn't, as well as the consequences of these actions, providing a technical analysis of system behavior.

One of the first mistakes, and regrets stated by Thread Zeppelin themselves, was not having Terry intervene in November once it became apparent that the project requirements were too vague, and access to resources would be limited, such as training the LLM on internal documents, or directly accessing the database. Thread Zeppelin did many things correctly, however, such as informing the client of the limitations of LLMs, as other people may have attempted it regardless, only to fall short in delivering their product at the end. In addition, by switching to a research project instead, they ensured that they would be able to deliver something, even if it isn't the originally intended product.

5.1 Root Cause

Section 5.1 addresses the primary and secondary causes for why things didn't go as planned, as well as how these issues came to fruition in the first place.

Primary Root Cause

The team struggled with a lack of access to additional resources, such as internal documentation to train the LLM, and direct access to the database. Despite asking the client for these resources, they were told they weren't allowed access to them, for privacy reasons. Thread Zeppelin only asked about these resources when they got to the point in their project where they

needed them, rather than asking about them in advance, which is what caused them to learn of this issue so late into the project development process.

Secondary Causes

The client's overestimation of LLM capabilities, expecting it to be able to do data analysis, when it actually can't, which an expert later confirmed. This expert was also only brought in at the end of February, which was extremely late for confirming the capabilities of LLMs, and whether they would be able to do what the client requested.

Why Safeguards Failed

Projects proposed by clients were not reviewed for whether they could be completed in the first place. Then, once the projects began, Thread Zeppelin did not adequately reach out to the course staff to voice their concerns, which in turn did not give the course staff the ability to intervene earlier on in the project's development. Had Thread Zeppelin brought up these issues earlier, they could have had more time to switch to another project, and by extension would have had more time to work on it.

5.2 Risks Identified

Section 5.2 discusses the risks that could have complicated this project. In particular, these risks are split into three categories. Technical risks are risks associated with things such as software bugs, hardware limitations, cybersecurity issues and the like. Process risks are flaws in how work is done, such as inadequate training, or failure to follow procedures. Lastly, operational risks are risks that relate to changes in requirements or regulations, as well as failures in people, systems, or processes.

Technical Risks

The use of an LLM trained on insufficient data can lead to poor performance when it comes to the summarizations the client was requesting. In addition, such an LLM is more likely to improperly handle sensitive data, raising concerns over security.

Process Risks

The constantly changing requirements, specifically asking for different file types, finally settling for .csv files, was one of the primary process risks, preventing Thread Zeppelin from moving forward with their project without knowing which requirement to fill. In addition, there was a contrast between the client's requirements and LLM capabilities, as the client wanted the LLM to do data analysis, when it wasn't actually capable of doing so. Lastly, there was only one excel file for training data, which the client wanted inputted raw into the LLM, even though it had some columns with missing labels.

Operational Risks

Due to the constantly changing file type requirements mentioned previously, Thread Zeppelin had less time to test whether the files were being outputted correctly. In addition, not having access to relevant training data, nor the database, makes it even less likely that the LLM functions properly when the client attempts to actually use it, as it would not have been trained on relevant data.

5.3 Follow-Up Actions

As the project has changed into a research project, the following steps are to properly research the capabilities of LLMs when it comes to summarizing manufacturing data. Doing so would not only provide Thread Zeppelin with advanced insight on LLM capabilities, but would also help provide the client with a greater understanding of what these models can do, as well as give them something to build off of when attempting to complete this project in the future.

Table 6 lays out the steps needed to be taken going forward, with their priority listed alongside it, in order of priority.

Table 6 — Follow-Up Action

Action	Priority
Discuss with the Client to see what they would like researched by each LLM	1
Decide on a method of how to test LLMs	2
Rank/sort each LLM based on their ability to complete the tasks they were asked to complete	3

6. Outstanding Tasks

This section describes all potential resolutions that Thread Zeppelin has in regards to creating a project to pass in for COS 497. This section will discuss four possible options, then provide insight on what PenUltimate believes to be the best solution.

Thread Zeppelin could rewrite their SRS, SDD, and CDRD. This would allow Thread Zeppelin the opportunity to go back and seek points of failure and determine the best route to fix these issues. Allowing them to come up with a solution to their problem while simultaneously allowing them to reflect on the problems that arose before. PenUltimate doesn’t recommend this

option due to the fact that this would be a repeat of what they did this previous semester and the only difference would be analyzing their management of the project and coming up with a solution to these problems.

Thread Zeppelin could rewrite their project proposal for a different project and subsequently write the SRS, SDD, UIDD, and CDRD. This would allow them to work on a project that is not possible for them to complete. PenUltimate doesn't recommend this option due to the fact that this would be a repeat of the previous semester for Thread Zeppelin.

The team could redo this project, this project may have been impossible to complete given the LLM restraints. If Thread Zeppelin didn't use the LLM they would be able to still achieve the desired output that their client specified. PenUltimate doesn't recommend this choice due to the fact that Thread Zeppelin is now weeks behind schedule because of these issues that have occurred. This is also making the assumption that it is possible to produce the desired output without the use of an LLM.

The final option, which PenUltimate recommends, is for Thread Zeppelin to write a research paper in regards to the LLMs they used for their project. Thread Zeppelin will be responsible for comparing these different LLMs as well as providing a potential resolution guide for their client. This guide will contain what Thread Zeppelin believes to be the best possible solution to fix the issue that they are not able to produce the desired output using an LLM.

Appendix A – Team Review Sign-off

All team members have signed to acknowledge they have reviewed this document and agreed on both its content and format. If team members have minor disagreements, they may state them in the comments area.

Rebecca Sonnemann

Signature: Rebecca Sonnemann (Mar 11, 2026 18:11:51 EDT) Date: 3/11/2026

Comments: _____

Brett Palmer

Signature: Brett Palmer (Mar 11, 2026 18:17:08 EDT) Date: 3/11/2026

Comments: _____

Monica Agneta

Signature: Monica Agneta (Mar 11, 2026 18:14:05 EDT) Date: 3/11/2026

Comments: _____

Ben Caras

Signature: Benjamin Caras (Mar 11, 2026 18:12:42 EDT) Date: 3/11/2026

Comments: _____

George Pitt

Signature: George Pitt (Mar 11, 2026 18:10:58 EDT) Date: 3/11/2026

Comments: _____

Appendix B – Document Contributions

Listed below are the contributions from each team member, and which sections of the document they worked on.

Brett

Brett wrote the entirety of section 1, wrote defects 2, 6, 8 and 9, and formatted the document. He served as the moderator of the post-mortem. Brett estimates that he did 20% of the work for this document.

Monica

Monica wrote the entirety of section 3, wrote defects 1 and 5, and created the table of contents. She served as recorder of the post-mortem. Monica estimates that she did 20% of the work for this document.

George

George wrote section 5 and defect 4. George served as an inspector of the post-mortem. George estimates that he did 20% of the work for this document.

Rebecca

Rebecca wrote the entirety of section 3, defects 7 and 11 of section 4.1, as well as the appendices. She served as reader of the post-mortem. Rebecca estimates she did 20% of the work for this document.

Ben

Ben wrote section 4, 4.2, and 6, and defects 3 and 10. He served as an inspector of the post-mortem. Ben estimates that he did 20% of the work for this document.